cs-mail: Communication Permission, Bonded Contact, and Protocol Settlement

Superseded design-history document. This file predates the current *cs-mail* whitepaper, protocol specification, Rust reference architecture, and deployment profile. It is retained for historical context only. Its per-attempt bonds, direct recipient collateral payouts, and funded user balances describe the earlier model, not the current design. Current documents use one charge per relationship request, request-linked refunds, and a company-owned member-distribution pool. The current protocol specification and C-SQD financial-program profile take precedence.

Abstract

cs-mail is a communication protocol organized around recipient-granted communication rights. A sender who has not been accepted by a recipient must post a refundable communication stake before sending a message. The stake contains a communication-processing charge and collateral. Acceptance refunds the full stake and establishes a standing permission to communicate without future bonds. Ignoring the message returns the collateral while transferring the processing charge to the recipient provider. Rejection transfers the collateral to the recipient and the processing charge to the recipient provider. The protocol therefore couples message delivery, communication permission, and financial settlement within a common state machine. This document develops the motivation, economic structure, protocol semantics, ledger model, and an initial path toward implementation and interoperability.

1 The Permission Problem in Electronic Communication

Electronic mail permits a sender who knows an address to attempt delivery to that address. The recipient may later classify the message as wanted or unwanted, and filtering systems may attempt to predict this classification before presentation. The underlying communication right remains weakly specified. Possession of an address is usually sufficient to impose at least some delivery, storage, filtering, and attention costs on the recipient or the recipient's provider.

This architecture worked tolerably when sending messages carried meaningful human and organizational costs. Automation has reduced those costs. Bulk mailing, automated sales systems, and generative systems allow a sender to create and transmit large quantities of individually plausible communication at low marginal cost. Filtering systems respond by estimating sender reputation, message quality, likely intent, and recipient interest. These estimates remain secondary judgments imposed on a communication system in which unsolicited transmission is inexpensive.

cs-mail gives the recipient an explicit role in establishing communication rights. A sender who has already been accepted may communicate without posting a bond. A sender who has not been accepted must place a refundable stake behind the attempt to establish contact. The recipient's response determines both the future communication relationship and the settlement of that stake.

The basic object of the protocol is therefore a directed relationship between sender and recipient. Message transmission occurs within the state of that relationship.

2 The Basic Mechanism

Consider a sender i and recipient j. If j has already accepted i, messages from i to j require no stake. If i has not been accepted, the sender must post a communication stake

$$B = C + S, \tag{1}$$

where C is a communication-processing charge and S is collateral.

In a federated network, these amounts are determined by the destination side. Let $p(j)$ denote the provider serving recipient j, let $\widehat{S}_j$ denote the recipient's collateral preference, and let $S^{\min}_{p(j)}$ denote any minimum collateral required by the recipient provider. The effective terms are

$$C = C_{p(j)}, \qquad S = \max\{\widehat{S}_j, S^{\min}_{p(j)}\}. \tag{2}$$

The recipient provider publishes these terms in a signed quote before the sender reserves the bond. The quote fixes C, S, the settlement unit, and the bond duration for that communication attempt. Neither the recipient nor the provider may change those terms after the bond has been reserved. The bond $C + S$ is the complete amount subject to the acceptance, rejection, and expiry transfers below; the protocol does not add a sender-provider charge, delivery fee, or clearing fee. Repeated attempts to contact the same unaccepted recipient may additionally be subject to temporary timing and liquidity constraints described later.

The recipient may accept the sender, reject the sender or message, or take no action before the bond expires. These outcomes settle the stake according to

$$\begin{aligned}
&\text{Accept} &&: \quad C + S \to \text{sender}, \\
&\text{Ignore} &&: \quad S \to \text{sender}, \qquad C \to \text{recipient provider}, \\
&\text{Reject} &&: \quad S \to \text{recipient}, \qquad C \to \text{recipient provider}.
\end{aligned} \tag{3}$$

Acceptance also changes the sender-recipient relationship. Future messages from that sender are delivered without additional stakes unless the recipient later revokes permission.

The economic role of C differs from the role of S. The processing charge compensates the network when an unsolicited communication attempt does not establish an accepted relationship. The collateral exposes the sender to a larger loss when the recipient affirmatively rejects the contact. A legitimate first contact can therefore be costless to the sender if the recipient accepts it.

3 Communication as a Directed Relationship

Let

$$A_{ij} \in \{\text{unknown}, \text{accepted}, \text{revoked}\} \tag{4}$$

denote the permission state governing communication from sender i to recipient j.

The relation is directional. The fact that j accepts messages from i does not imply that i has accepted messages from j. Each recipient controls incoming communication independently.

The state determines whether a new bond is required:

$$\text{bond_required}(i, j) = \begin{cases} 0, & A_{ij} = \text{accepted}, \\ 1, & A_{ij} \in \{\text{unknown}, \text{revoked}\}. \end{cases} \tag{5}$$

Acceptance changes A_{ij} to `accepted`. Revocation changes it to `revoked`. A revoked sender may attempt contact again by posting a new bond.

This structure gives the protocol a persistent memory of communication permission. The bond is primarily associated with crossing an unaccepted relationship boundary. Once the boundary has been crossed successfully, ordinary communication proceeds without recurring financial friction.

4 Why the Stake Is Refundable

A fixed fee on every message would price communication itself. That would impose continuing costs on ordinary relationships and would make high-volume legitimate communication expensive. *cs-mail* instead places an economic burden on attempts to communicate before a relationship has been accepted.

Suppose an unknown sender posts a one-dollar stake. If the recipient accepts the sender, the entire dollar returns to the sender. The first contact temporarily consumes liquidity but does not produce a lasting payment. A sender who makes highly selective contact attempts can therefore communicate at low expected cost if recipients commonly accept those attempts.

A sender who transmits indiscriminately faces a different distribution of outcomes. Ignored messages lose C. Rejected messages lose both C and S. Large-scale unsolicited communication also requires the sender to reserve capital while bonds remain unsettled.

The resulting cost depends on the recipient response to the attempted relationship. The protocol therefore creates a direct economic connection between the sender's choice to initiate contact and the recipient's judgment of that contact.

5 Acceptance, Rejection, and Silence

Acceptance, rejection, and silence carry different information.

Acceptance is an explicit grant of future communication permission. The sender receives the full stake back, and the relationship enters the accepted state.

Rejection is an explicit negative judgment. The recipient has determined that the unsolicited contact should not establish a communication relationship. The collateral S is transferred to the recipient. The processing charge C is transferred to the recipient provider.

Silence is ambiguous. A message may remain unanswered because the recipient was uninterested, unavailable, inattentive, undecided, or simply unwilling to make an explicit judgment. *cs-mail* therefore does not treat silence as equivalent to rejection. On expiry, the collateral returns to the sender while the communication-processing charge remains payable to the recipient provider.

This gives silence an economic cost without assigning it the same meaning as an affirmative rejection. It also avoids paying recipients merely for neglecting unsolicited messages. Recipient compensation requires an explicit rejection event.

6 Repeated Unaccepted Contact Attempts

The basic bond governs a single attempt to establish contact. A sender may nevertheless make repeated attempts after earlier messages are ignored or rejected. Because S returns to the sender after silence, a persistent sender might treat repeated losses of C as a cost of searching for eventual acceptance.

cs-mail constrains this behavior without creating another permanent charge or another beneficiary of failed communication. Repeated attempts by the same verified sender principal to the same

recipient protocol identity become progressively less frequent. Let P_i denote C-SQD's private principal identifier for sender i, and let I_j denote the recipient's public protocol identity. If k denotes the current attempt level for the directed pair (P_i, I_j), a new attempt may not be admitted until a backoff interval Δ_k has elapsed, where

$$\Delta_{k+1} \geq \Delta_k. \tag{6}$$

Repeated attempts may also require a refundable persistence reserve L_k. The total amount reserved for an attempt is then

$$R_k = B + L_k = C + S + L_k, \tag{7}$$

with

$$L_{k+1} \geq L_k. \tag{8}$$

For an initial attempt, $\Delta_0 = 0$ and $L_0 = 0$, so the ordinary requirement remains $R_0 = B = C + S$. The persistence reserve is distinct from both C and S. It is never transferred to the recipient or either provider. Acceptance returns it immediately. After rejection or expiry it remains unavailable until its specified release time and then returns to the sender.

The attempt level advances when the recipient provider admits an unaccepted message, rather than when the message later settles. This prevents a sender from submitting many low-level attempts before the first outcome is known. Acceptance resets the attempt level and releases the relationship's active persistence reserves. A message that is never admitted because of technical nondelivery neither advances the level nor retains a persistence reserve.

Backoff constrains the rate of repeated attempts, while the persistence reserve constrains the capital required to sustain them. The schedules for Δ_k and L_k are protocol-defined, and their effective values are disclosed in the signed contact terms. Because both are keyed internally to (P_i, I_j), changing the sender's address, operational key, or provider does not reset the history. Distinct recipient protocol identities remain distinct communication boundaries.

7 Protocol State Machine

A communication attempt from an unaccepted sender creates a pending bond and a pending message relationship. At attempt level k, the transition can be represented as

$$\text{Unaccepted} \xrightarrow{\text{wait } \Delta_k,\ \text{reserve } R_k,\ \text{send}} \text{Pending}. \tag{9}$$

The pending state admits three terminal settlement outcomes:

$$\text{Pending} \xrightarrow{\text{accept}} \text{Accepted}, \tag{10}$$

$$\text{Pending} \xrightarrow{\text{reject}} \text{Rejected}, \tag{11}$$

and

$$\text{Pending} \xrightarrow{\text{expiry}} \text{Ignored}. \tag{12}$$

Acceptance changes the persistent sender-recipient permission state. Rejection and expiry leave the sender outside the accepted set. Revocation later changes an accepted relationship back to a bond-required state:

$$\text{Accepted} \xrightarrow{\text{revoke}} \text{Revoked}. \tag{13}$$

A new communication attempt from a revoked sender requires a fresh bond.

The financial transition and permission transition should occur atomically when they are coupled. An acceptance event should not be able to establish permission while leaving the bond unsettled. A rejection should not transfer collateral while leaving the bond available for a second settlement.

8 Protocol Objects

A minimal implementation requires a small set of persistent protocol objects.

8.1 Identity

The initial C-SQD deployment should distinguish a privately verified principal from the public protocol identities through which that principal communicates. For an individual, C-SQD may establish the principal using full legal name, date of birth, a verified bank account, address, and phone; the name, address, and phone must match what the bank reports for the account's owner. An enrollment whose evidence shares the bank account, or any two of these fields, with another person's is held until C-SQD reviews it. For a corporation or institution, C-SQD may use legal name, jurisdiction, EIN or another legal-entity registry identifier, a verified institutional bank account, and evidence that the enrolling representative is authorized to act for the entity.

After verification, C-SQD assigns an immutable internal principal identifier, denoted P_i. The identity evidence and P_i remain private to C-SQD. They are not displayed to users or included in ordinary messages.

A public email address or equivalent address is a protocol identity, denoted I_i. Users grant and revoke communication permission with respect to these visible protocol identities. One principal may control multiple protocol identities, and each protocol identity may authorize one or more rotatable operational keys. Key rotation, account recovery, and provider migration can therefore occur without changing the underlying principal.

Permission state and abuse-control state use these layers differently. Acceptance remains a directed relationship between visible protocol identities, so accepting one sender address does not automatically accept every address controlled by the same principal. Repeated-attempt state instead follows the private sender principal, preventing a sender from resetting backoff by changing addresses, keys, or providers.

8.2 Contact Relationship

A contact relationship records the directed communication permission between two identities. It minimally contains

- sender identity,
- recipient identity,
- permission state,
- state version or sequence number,
- time of the most recent transition.

The state should be derived from authenticated protocol events rather than arbitrary database mutation.

8.3 Repeated-Attempt State

Repeated-attempt state is separate from the visible contact relationship. It is indexed internally by the sender principal P_i and recipient protocol identity I_j. It minimally contains

- private sender-principal reference,
- recipient protocol identity,
- attempt level k,
- earliest time at which another unaccepted attempt may be admitted,
- active persistence-reserve references,
- state version or sequence number,
- time of the most recent transition.

This separation permits multiple email identities controlled by the same principal to remain distinct for recipient permission while sharing the sender's anti-churn history with respect to a particular recipient identity.

8.4 Message Envelope

The message envelope should identify the sender, recipient, message identifier, creation time, content reference, and any bond associated with delivery. The recipient or recipient provider must be able to verify that a required bond has been reserved before accepting the message into the protocol.

8.5 Contact Terms

Before creating a bond, an unaccepted sender obtains signed contact terms from the recipient provider. These terms minimally contain

- sender protocol identity,
- recipient identity,
- recipient-provider identity,
- an opaque reference to the applicable repeated-attempt state,
- communication-processing charge C,
- effective collateral S,
- any persistence reserve L_k and its release conditions,
- attempt level k and earliest admission time,
- repeated-attempt state version,

- currency or settlement unit,
- bond duration,
- quote expiry time,
- policy version,
- provider signature.

The signed terms expose the effective collateral required for the attempt; they need not reveal whether that amount arose from the recipient's preference or the provider minimum. A quote must be valid when the bond and any persistence reserve are created. The resulting records incorporate the quoted terms, which remain fixed for their full lifecycles. Later policy changes apply only to future attempts.

8.6 Bond

A bond should contain at least

- a unique bond identifier,
- sender and recipient identities,
- the message or communication attempt to which it applies,
- C,
- S,
- currency or settlement unit,
- creation time,
- expiry time,
- current bond state.

A bond may pass from `reserved` to exactly one settlement state: `accepted`, `rejected`, or `expired`.

8.7 Persistence Reserve

A repeated attempt may also create a persistence reservation containing the private sender-principal reference, recipient protocol identity, communication attempt, L_k, its creation time, its scheduled release time, and its current state. It is released immediately upon acceptance or technical nondelivery. Following rejection or expiry, the ledger creates an idempotent deferred release returning L_k to the sender at the specified time. The persistence reserve is never transferred to a recipient or provider.

8.8 Protocol Events

State changes should be expressed as authenticated events. An initial event vocabulary may include

- bond reservation,
- persistence reservation and release,
- repeated-attempt admission and reset,
- message delivery,
- sender acceptance,
- sender rejection,
- bond expiry,
- sender revocation.

These events form the basis for deterministic state transition and ledger settlement.

9 The Financial Ledger

The financial ledger is part of *cs-mail* at the level of protocol semantics. Message validity and bond settlement depend on financial state. A recipient provider must be able to determine whether a sender has successfully reserved the required bond and any persistence reserve. Acceptance, rejection, and expiry must create deterministic settlement consequences.

Three layers should remain conceptually distinct:

$$\boxed{\text{protocol ledger semantics} \neq \text{ledger operator} \neq \text{banking rails}} \tag{14}$$

Protocol ledger semantics define the meaning of balance reservation, bond creation, settlement, release, and transfer. The ledger operator maintains the canonical record of those obligations. External financial rails move funds into and out of the ledger.

This separation permits a large number of communication settlements to occur internally without requiring a corresponding card, ACH, or bank transaction for every message. Users may fund their balances occasionally. Protocol events then reserve, release, and transfer internal balances according to the defined settlement rules.

A minimal ledger interface might contain operations corresponding to

- reserve an amount against a bond,
- reserve and later release persistence collateral,
- release reserved funds after acceptance,
- transfer the processing charge to the recipient provider,
- transfer collateral to the recipient after rejection,
- return collateral to the sender after expiry,
- query available and reserved balances.

The ledger should be append-only at the journal level. Derived balances can be materialized for performance, while the underlying history remains auditable.

10 Initial C-SQD Deployment

An initial deployment can centralize protocol operation while preserving logical separation among protocol roles. C-SQD can operate

- the reference *cs-mail* implementation,
- the identity system,
- the canonical contact-permission state,
- the canonical bond ledger,
- the initial message-delivery infrastructure,
- the clearing logic for all internal settlements.

External financial providers can handle account funding, withdrawals, card processing, ACH, and other regulated payment functions. This keeps the communication ledger under C-SQD control while avoiding the requirement that C-SQD initially build or directly operate all underlying financial rails.

The protocol should nevertheless define its state and event semantics independently of the initial C-SQD deployment. This allows later implementations to reproduce the same behavior.

11 Interoperable Providers

A later *cs-mail* network may contain multiple providers. A sender could maintain an account with one provider while the recipient uses another. The sender provider would certify that the required bond has been reserved. The recipient provider would verify the reservation before accepting the message.

The resulting inter-provider obligations could be cleared periodically. If Provider A owes Provider B ten thousand dollars in net recipient collateral and Provider B owes Provider A eight thousand dollars, the providers may settle the two-thousand-dollar net difference through an external rail while retaining the individual protocol events in their ledgers.

Before an unaccepted sender reserves funds, the sender provider obtains a signed contact-terms quote from the recipient provider and presents the complete requirement $R_k = C + S + L_k$ to the sender. The recipient provider verifies its quote, the bond, any persistence reserve, and the current repeated-attempt state before admitting the message. Inter-provider clearing implements the transfer obligations created by C and S. The persistence reserve L_k remains the sender's property and is never transferred to either provider. Neither clearing nor persistence reservation creates an additional protocol fee.

In the initial C-SQD deployment, C-SQD resolves each authenticated sender identity to its private principal record directly. In a later federated deployment, another provider should receive only an opaque C-SQD-signed assertion sufficient to address the same sender-principal/recipient-identity attempt state. Full names, dates of birth, bank information, EINs, legal-entity records, and C-SQD's internal principal identifiers are not disclosed through ordinary protocol messages.

Interoperability therefore requires standard representations for

- identities,

- provider identity,
- message envelopes,
- bond commitments,
- repeated-attempt state and persistence-reserve commitments,
- authenticated recipient decisions,
- settlement receipts,
- provider-to-provider clearing obligations.

The protocol can specify these objects without requiring every provider to use the same database, internal ledger software, or banking partner.

12 Compatibility with Conventional Email

cs-mail can be introduced without requiring immediate replacement of SMTP. A compatibility gateway can receive messages from conventional email senders and translate them into a *cs-mail* contact attempt.

If the legacy sender is already associated with an accepted identity or an explicit recipient-granted capability, the gateway may deliver normally. If the sender is unaccepted, the gateway can return a payment challenge or provide a web-based flow for posting the required stake. Once funded, the gateway creates the corresponding *cs-mail* bond and message envelope.

Outgoing messages from *cs-mail* users to conventional email recipients can similarly pass through an SMTP gateway. These messages would not receive the full permission and settlement semantics unless the recipient participates in *cs-mail*.

The gateway is therefore migration infrastructure. Native *cs-mail* communication can use cryptographic identity, direct permission state, and ledger settlement without depending on SMTP conventions.

13 Incentives and Abuse Resistance

The protocol creates several forms of cost for unsolicited communication.

First, ignored messages incur the processing charge C. Large-scale outreach with low recipient response therefore produces direct expected cost.

Second, rejected messages lose both C and S. Senders whose messages are frequently judged unwanted face a larger expected loss.

Third, unsettled bonds tie up sender liquidity. Even when collateral is eventually returned after silence, a sender attempting millions of simultaneous contacts may need to reserve a substantial balance until those contacts expire.

Fourth, accepted relationships remove recurring bond costs. Communication among recognized contacts therefore remains inexpensive and operationally simple.

Fifth, repeated attempts to the same unaccepted recipient encounter increasing backoff and refundable persistence-reserve requirements. These controls make continued attempts slower and more capital-intensive without increasing recipient or provider revenue. Acceptance removes both constraints by resetting the attempt level.

The allocation of pricing authority follows the distinct purposes of the two components. The recipient provider establishes C and may establish a minimum S needed to protect the service from abuse. The recipient chooses a collateral preference at or above that minimum. A higher S may discourage unwanted communication, but it may also discourage legitimate first contact. Interfaces should therefore make the effective contact terms clear before a sender commits funds.

The protocol also limits some recipient-side incentives. Ignoring messages does not transfer collateral to the recipient. A recipient must actively reject a message to receive S. This reduces the value of passively accumulating unsolicited messages for financial gain.

Recipient-provider incentives require explicit governance. Because the recipient provider receives C when communication attempts fail to become accepted relationships, acceptance should be easy to perform and settlement rules should be auditable. The protocol should define the settlement consequences independently of interface design or ranking algorithms.

14 Scoped and Transactional Communication

Some communication does not fit naturally into a permanent personal contact relationship. Banks, merchants, airlines, healthcare organizations, government agencies, and automated services may need to send messages within a narrow context.

cs-mail can support recipient-issued communication capabilities. A recipient might authorize an organization to send messages associated with a transaction, account, appointment, purchase, or time interval. Such a capability can exempt matching communication from bonds without granting unrestricted standing permission.

A capability may specify

- authorized sender identity,
- recipient identity,
- permitted purpose or message class,
- validity interval,
- maximum number or rate of messages,
- revocation conditions.

These capabilities extend the same permission model while preserving narrower recipient control.

15 Historical Reference Settlement Semantics

The earlier draft proposed the following rules. They are historical and are not conformance requirements for the current request model.

Contact-Term Discovery and Fixation

Before reserving funds, an unaccepted sender must obtain a valid signed quote from the recipient provider. The quote specifies C, S, any applicable L_k, the attempt level and earliest admission time, the settlement unit, the bond duration, the persistence-release conditions, and a policy and state version. The sender must be shown the complete requirement $R_k = C + S + L_k$ before authorizing

reservation. A bond and persistence reserve may be created only while the quote remains valid. Once reserved, the quoted terms are immutable for those records; changes in recipient or provider policy apply only to later communication attempts. The refundable persistence reserve is not a fee, and no additional sender-provider charge, delivery fee, or clearing fee forms part of the protocol settlement.

Accepted Sender

If $A_{ij} =$ accepted, sender i may send recipient j without posting a bond, subject to ordinary protocol limits and any narrower recipient policies.

Unaccepted Sender

If $A_{ij} \neq$ accepted, a message requiring ordinary unsolicited delivery must reference a valid reserved bond with value

$$B = C + S. \tag{15}$$

At attempt level k, the message must also reference any required persistence reserve L_k, giving a total reservation

$$R_k = C + S + L_k. \tag{16}$$

The message must not be accepted as valid native *cs-mail* delivery unless the required reservations and current repeated-attempt state can be verified.

Repeated-Attempt Admission

C-SQD resolves the authenticated sender protocol identity to its private verified principal before evaluating repeated-attempt state. That state is indexed by the sender principal P_i and destination protocol identity I_j, rather than by a replaceable sender address, operational key, or provider account. A new sender identity associated with the same principal therefore does not create fresh attempt history. Distinct destination identities retain separate histories. For the initial attempt, $k = 0$, $\Delta_0 = 0$, and $L_0 = 0$. At level k, an attempt may be admitted only after its quoted eligibility time and only if $C + S + L_k$ has been reserved.

Admission and advancement of the repeated-attempt state must occur atomically. Once an unaccepted message is admitted, the state advances to the next attempt level and records the next eligibility time according to the protocol backoff schedule. A provisional reservation that never produces admitted delivery because of technical failure is released and does not advance the state. Rejection and expiry preserve the advanced level. Acceptance resets it.

Acceptance

An authenticated acceptance event generated by the recipient causes

$$A_{ij} \leftarrow \text{accepted} \tag{17}$$

and

$$C + S \rightarrow \text{sender}. \tag{18}$$

The bond enters a terminal accepted state. The repeated-attempt level for the corresponding sender-principal/recipient-identity pair (P_i, I_j) resets to zero, its backoff is removed, and all active persistence reserves for that pair are released immediately to the sender. Acceptance applies to the visible sender protocol identity that the recipient accepted; it does not automatically grant permission to other identities controlled by the same principal.

Rejection

An authenticated rejection event generated by the recipient causes

$$C \rightarrow \text{recipient provider} \tag{19}$$

and

$$S \rightarrow \text{recipient.} \tag{20}$$

The sender remains outside the accepted set. The bond enters a terminal rejected state. Any L_k associated with the attempt remains locked until its quoted release time and is then returned to the sender; it is not transferred to the recipient or either provider.

Expiry

If the bond expires without a valid acceptance or rejection event,

$$C \rightarrow \text{recipient provider} \tag{21}$$

and

$$S \rightarrow \text{sender.} \tag{22}$$

The sender remains outside the accepted set. The bond enters a terminal expired state. Any L_k associated with the attempt remains locked until its quoted release time and is then returned to the sender; it is not transferred to the recipient or either provider.

Revocation

An authenticated revocation event generated by the recipient causes

$$A_{ij} \leftarrow \text{revoked.} \tag{23}$$

Revocation does not alter previously settled bonds. Future messages again require a bond unless another valid capability authorizes delivery.

Settlement Finality

Each bond may enter exactly one terminal settlement state. A terminal bond cannot be accepted, rejected, expired, refunded, or transferred again. Each persistence reserve may likewise be released exactly once, either immediately after acceptance or nondelivery, or at its scheduled release time following rejection or expiry. Settlement operations and their corresponding protocol-state transitions must be atomic from the perspective of protocol validity.

16 Implementation Direction

A reference implementation should be organized around a small deterministic protocol kernel. The kernel defines protocol objects, validates commands and authenticated events, and computes state and ledger transitions. It should not perform networking, database access, external payment processing, random-number generation, or direct system-clock reads. Time, signatures, and prior state should instead be explicit inputs, so that the same valid input always produces the same transition.

The implementation can be separated into Rust crates or modules such as

```
cs-mail-core
    private principals
    protocol identities
    operational keys
    contacts
    contact terms
    repeated attempts
    messages
    bonds
    events
    transitions
    invariants

cs-mail-ledger
    accounts
    reservations
    journal
    settlement
    deferred releases
    balance projections

cs-mail-crypto
    keys
    signatures
    verification

cs-mail-wire
    versioned protocol representations
    serialization
    compatibility validation

cs-mail-storage
    event journal
    snapshots and projections
    transactional outbox

cs-mail-content
    message bodies
```

```
    attachments
    content references

cs-mail-server
    protocol API
    authentication
    persistence
    delivery
    expiry processing

cs-mail-smtp-gateway
    inbound compatibility
    outbound compatibility
    bond challenges

cs-mail-client
    inbox
    contacts
    accept/reject controls
    wallet
```

The ledger should use double-entry accounting with an append-only journal. Available and reserved balances may be maintained as materialized projections, but journal entries remain the authoritative financial history. Commands and settlement operations must be idempotent so that retries or replayed events cannot transfer or release the same value twice. Permission changes and their corresponding financial settlements must be atomic from the perspective of protocol validity.

The core should enforce at least the following invariants:

- each bond enters at most one terminal settlement state;
- ledger value is conserved across every transition;
- a message from an unaccepted sender requires a valid signed quote, a bond of exactly $C + S$, and any applicable persistence reserve L_k;
- repeated-attempt admission atomically advances sender-principal/recipient-identity backoff state;
- acceptance establishes permission, refunds exactly $C + S$, releases active persistence reserves, and resets the attempt level;
- rejection and expiry implement exactly the settlement rules defined by the protocol;
- persistence reserves are returned exactly once and are never transferred to a recipient or provider;
- accepted communication requires no bond and incurs no protocol message fee;
- duplicate commands and replayed events cannot produce duplicate settlement.

Domain types should remain independent of any particular database or wire encoding. A versioned wire representation can support JSON, CBOR, Protobuf, or another interoperable encoding without making that representation the internal state model. Message bodies and attachments should likewise remain separate from the small protocol envelope and strongly consistent ledger state. Content storage and search indexes can be distributed and updated asynchronously without weakening bond validity or settlement finality.

The logical state machine is naturally partitionable. Permission and message state can be sharded by recipient protocol identity, while repeated-attempt state can be sharded by the sender-principal/recipient-identity pair and account and reservation state can be partitioned according to ledger ownership. Event sourcing should use partitioned streams and materialized projections rather than replaying a global history. Repeated-attempt state must survive changes to the sender's protocol identity, operational keys, and provider. Relationship state should normally remain passive and be loaded on demand; an implementation should not allocate a permanently running actor or task to every possible sender–recipient pair.

Implementation should proceed in stages:

1. implement the complete deterministic lifecycle of a bonded message, including repeated-attempt backoff and persistence reservation, in memory;
2. test transition sequences and invariants with unit and property-based tests;
3. add the append-only double-entry journal and idempotent settlement;
4. add signed identities, authenticated events, and signed contact-term quotes;
5. persist events and maintain snapshots and materialized projections;
6. add delivery APIs, content storage, and expiry processing;
7. add SMTP compatibility; and
8. add provider federation and clearing only after the centralized semantics are stable.

The first milestone should therefore accept two public protocol identities associated with privately verified C-SQD principals and a funded sender account. It should execute contact-term quotation, sender-principal resolution, eligibility checking, bond and persistence reservation, message admission, acceptance, rejection, expiry, deferred release, attempt-level reset, settlement, and later revocation entirely through deterministic transitions. Persistence, transport, compatibility gateways, and federation can then be layered around that tested kernel without changing its semantics.

17 Conclusion

cs-mail treats communication permission as persistent protocol state. Accepted senders communicate freely. Unaccepted senders post a refundable communication stake. Recipient decisions determine both future permission and stake settlement.

The mechanism gives distinct meanings to acceptance, rejection, and silence. It creates economic costs for unsuccessful unsolicited communication while preserving costless protocol communication within accepted relationships. In a federated network, recipient providers publish binding contact terms: the provider establishes C, the recipient selects a collateral preference S subject to any provider minimum, and the complete bond $C + S$ is fixed before sending. Repeated attempts by the

same privately verified sender principal to the same recipient protocol identity encounter increasing backoff and refundable persistence-reserve requirements, both of which disappear upon acceptance without creating another beneficiary of failed communication. Users continue to grant permission to visible email identities, and no private identity evidence is disclosed through ordinary messages. No additional message or clearing fee is added by the protocol. The ledger is part of the protocol because financial reservation and settlement determine whether a bonded message is valid and how recipient actions change ownership of the stake. External financial rails can remain independent of the protocol and may be provided by regulated third parties.

An initial C-SQD deployment can operate the reference implementation and canonical ledger centrally. The protocol can still be specified in a form that permits later provider interoperability, shared clearing, and independent implementations. This provides a path from a centralized product to a broader communication network without changing the basic semantics of permission and settlement.